ResearchAgent



Research with a wallet.

ResearchAgent answers from free sources first, then pays writers for the articles it is still missing, **over x402 on the XRP Ledger**, inside a budget you set. A model recommends; only policy code can pay.

Try it live	https://ra-front-651013803640.asia-southeast1.run.app · Shared XRPL Testnet sandbox · no real value · open 13–19 Nov
Watch a run	collaboration95.github.io/theFastandtheFungible/demo/ · a recorded Testnet run, any time

```
$ npx -y https://github.com/Collaboration95/theFastandtheFungible/releases/download/v1.0.2/researchagent-mcp-1.0.2.tgz try --testnet
```

- XRPL Testnet · no real value · fictional writers, plus our own labelled build notes · TEST-SGD stands in for RLUSD

— Progress since SingHacks · 19–20 Sep 2026

From a hackathon prototype to a product anyone can check on the ledger

At SingHacks a language model proposed each purchase and a person approved every quote before payment; payment was a fixture by default, with optional Testnet settlement.

	At SingHacks	Now
Who decides to buy	A language model proposed; a person approved every quote	Policy code buys within your budget after one plan confirmation; a decision model only scores
Fair exchange	Pay first, then read	Key-reveal escrow: the writer is paid only by publishing the article’s key
Refunds	None	A broken promise is refunded on the ledger
Proof	Citations checked inside the app	An offline verifier anyone can run, in TypeScript or Python
Ways in	One web page	A live sandbox, an MCP server for Claude Code and Codex, SDKs and a Mac app

236
pull requests merged since 21 Sep

13
releases, v0.1.0 to v1.0.2

61
Testnet transactions you can re-check live on our ledger page

224
receipt test vectors; two independent verifiers agree on all

Counted at 1e2958ce on 11 Nov 2026 (git and gh queries in site/progress/data.json) · “At SingHacks” is the 19–20 Sep event build (e16435a3)

One line runs the whole money path on real Testnet

```
$ npx -y https://github.com/Collaboration95/theFastandtheFungible/releases/download/v1.0.2/researchagent-mcp-1.0.2.tgz try --testnet
```

```
Testnet · no real value · Fixture models and fictional writers
1. Create: 04900C76…16DB031C
2. Finish · sha256(K) = Condition ✓: E95BED8E…FABF6EAA
3. Refund · writer-signed · fixture writer cooperates (a real writer could refuse): B248161A…F785E2BB
4. Re-buy create: 2F5C3D75…2D9FD1FF
5. Re-buy finish · sha256(K) = Condition ✓: 528DFA6F…92CDC389
6. Anchor: 7D5B506B…A6DBCEE7
VERIFY c349c3ec-ae93-4c2e-9bfd-190aa146bd6b
PASS CONTRIBUTIONS 1/1
PASS MANIFEST   7/7
PASS PASSAGES   7/7
PASS CITATION   16/16
PASS GRANT      1/1
PASS RECEIPT    3/3
PASS REFUND     2/2
PASS AUDIT CHAIN 1/1
PASS testnet 12/12 (read-only, s.altnet)
SKIP writer credentials: Unsigned advisory identity; requires online check
SKIP historical origin: v1 origin unresolved; original singleton semantics
PASS audit anchor: Validated XRPL Testnet ledger 22132261
Finished in 69.0 s · budget S$2.00 · net S$0.40 · gross S$0.70
RESULT PASS
```

84.2 s

from an empty npm cache to six Testnet links and RESULT PASS: buy, refund a broken promise, re-buy, anchor, verify. One cold run on one Mac.

2 of 4

cold runs overran the 90 s target. We keep the misses.

If the faucet fails, it prints a labelled SIMULATED run and the reason. The output here is a committed fresh-state run; its own clock excludes the npm download. Links are shortened to their hashes.

One question, one budget, and only code can spend it

1
Ask
A question and a budget you set.

2
Confirm the plan
A short sheet confirms the plan, not a purchase.

3
Search writers
Writers return abstracts and signed promises, never premium text.

4
Decide
A decision model scores what is worth buying, weighted by trust.

5
Pay
Policy code locks payment in a key-reveal escrow on XRPL.

6
Verify and answer
Decrypt, check every promise, refund a broken one, cite, anchor.

Five hard gates, on every run

1. No premium bytes before a grant.
2. The budget is the only spending authority; only policy code can buy, never the language model.
3. One charge per intent, across retries and restarts.
4. Every citation points at a delivered passage.
5. Anything simulated or substituted is labelled.

0 gate violations

across the v5 held-out and off-topic question banks on Testnet (11 Nov).

Spending caps for MCP clients default to zero, and the calling model cannot raise them.

— Creativity

The writer is paid only by publishing the key: delivery and payment are one ledger event

Writer

Encrypt and promise

A fresh key K per buyer. Signs the quote: `sha256(K)`, ciphertext hash, passage manifest, price.

Buyer policy

EscrowCreate

Native XRPL Escrow. Condition = `sha256(K)`, with a CancelAfter deadline.

Writer

EscrowFinish

To be paid, the writer must reveal K as the fulfilment. K is now public on the ledger.

Buyer policy

Decrypt and check

Decrypt, check every promised passage, then grant and cite.

Promise kept	Create →	Finish, K revealed →	decrypt ✓ passages ✓ →	cited	`2F5C3D75…` → `528DFA6F…`
Promise broken	Create →	Finish, K revealed →	claim missing →	writer-signed refund + fraud proof	`E95BED8E…` → `B248161A…`
Writer no-show	Create →	no key by CancelAfter →	EscrowCancel: money back, nothing granted		`8DA4ECE7…` → `F89DD9B1…`

> **What the ledger does and doesn't enforce.** Key reveal makes **delivery** atomic: no key, no payment. **Quality** rests on the signed promises, a fraud proof anyone can check, and reputation. After a finish, the refund is writer-signed: our fixture writer cooperates, and a real writer could refuse. No smart contract: Escrow with a PREIMAGE-SHA-256 condition is native XRPL.

The same key-reveal escrow, settled in a stablecoin-style token

TEST-SGD through XLS-85 token escrow

- **Opt in** with `--iou` on the MCP server or the web app.
- **Preflight refuses** a missing issuer locking permission, frozen or unauthorized trust lines, short balances and transfer fees.
- **Both verifiers** check currency, issuer and the writer's trust-line balance change, because EscrowFinish has no delivered_amount.

`create E36AF9F1…18878C5C · finish 384CA34F…181C02F5 · anchor 8174E3BE…8ADC35A3`

Official RLUSD: one issuer flag away

- **Payments work.** Official Testnet RLUSD paid a writer on 3 Nov (`1F2449A4…`), and a broken promise was refunded.
- **Escrow is blocked by the issuer, not the network.** Token escrow is enabled on Testnet, but the RLUSD issuer's flags lack AllowTrustLineLocking. One AccountSet by the issuer would allow it.
- **So we escrow our own token** and say so on every amount, receipt and ledger row. It is our top builder-feedback finding.

● TEST-SGD · an IOU from our Testnet issuer · no real value · stand-in for RLUSD · no peg, backing or mainnet claim

— Technical depth

Don’t trust our agent. Verify it, against the ledger.

```
# the Python verifier: standard library only, no Node
$ researchagent verify UC3.receipts.json --ledger
PASS checks: 57 PASS, 0 FAIL, 2 SKIP
PASS audit anchor: Validated XRPL Testnet ledger 22057921
PASS testnet 12/12 (read-only, s.altnet)
RESULT PASS
```

Any receipt the app exports can be checked the same way, by anyone, offline or against the ledger.

- **Two independent verifiers**, TypeScript and Python, agree on all 224 shared vectors: 19 must pass and 205 tampered ones must fail.
- **61 Testnet transactions** are re-checked live, read-only, every time our public ledger page loads.
- **Every finished answer is anchored:** an AccountSet whose memo is the audit-chain head, even when nothing was bought.
- **Persist before submit.** A restart reconciles the same signed transaction and never signs twice; a missing one is *pending*, not failed.
- **Exact delivery.** Partial payments are refused before submit; amount, currency and issuer are checked after.
- **Prompt injection can’t spend:** model inputs are untrusted, and only policy code can buy.

Output: docs/evidence/round4-j2/python-verifier.txt on docs/evidence/lane-b3/UC3.receipts.json · tests/fixtures/receipt-conformance/ · the /ledger/ page

— Reachability

Every way in runs from one package, for people and for other agents

In the browser

The live sandbox: ask your own question, nothing to install.

Shared XRPL Testnet sandbox · no real value · open 13–19 Nov

Claude Code, any MCP client

`claude mcp add … --testnet`. Spending caps default to zero; the calling model can't raise them.

Cold start 6.3 s · 3.4 MB package

Local app, no clone

`app --testnet` serves the real app on 127.0.0.1 with faucet wallets.

Ready in 27.0 s from an empty cache

TypeScript and Python SDKs

Ask, stream, verify. No purchase method: only backend policy can buy.

The Python verifier needs no Node

Writers in one command

`publish ./notes --testnet`: faucet wallet, XRPL Credential, key-reveal escrow.

33.9 s from `init` to a validated Credential

Hire the agent over x402

Another agent pays ResearchAgent; it buys, pays every cited writer a royalty and returns the change.

Recorded on Testnet, 3 Nov

> **Does it buy the right thing?** v5 blind held-out: 16/20 bought and cited, 0/10 off-topic purchases (run once, 11 Nov; fixture decision model, team-authored writers, XRPL Testnet). The key fact leads 7/20 answers. Packages: GitHub Releases; not yet on npm, PyPI or the MCP Registry.

docs/install.md · docs/evidence/round4-j2/ (timings) · round4-j3/slim-onboarding.json · hire-274.md · v4-heldout/README.md

— Feasibility

Fractions of a cent per purchase, and it recovers from crashes

435 drops

network fees per key-reveal purchase

12 to create the escrow and 423 to finish it, because the finish carries the key: 0.000435 XRP, measured on Testnet.

0 double charges

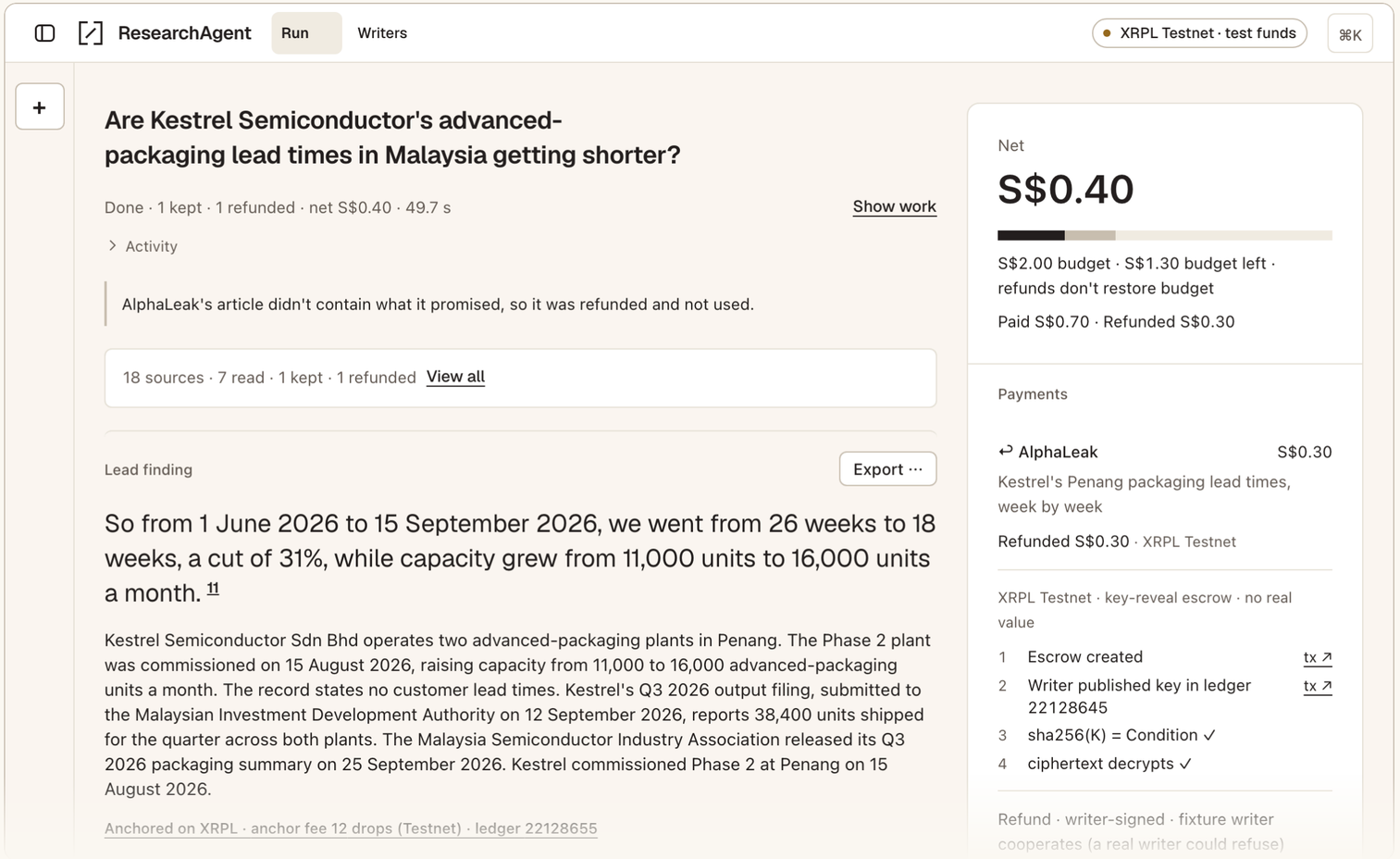
under chaos, across retries and restarts

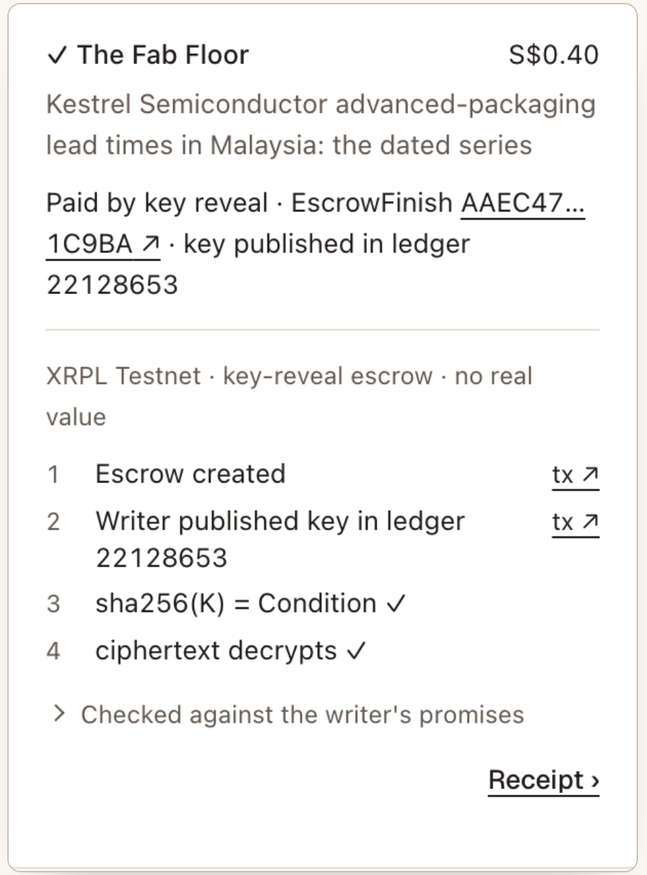
500 seeded runs with 130 real process kills; the reconciler recovers in-flight payments. Synthetic ledger, 3 Nov.

How it runs

- **Hosted for judges on Cloud Run,** 13–19 Nov: one shared Testnet payer, caps per visitor, owner controls off.
- **Persist before submit:** a restart never signs twice; a missing transaction is pending, never failed.
- **Each answer names its models:** live, or a labelled fixture fallback.
- **Mainnet is disabled in code.** What it needs is on the compliance slide and in our builder feedback.

The money is the hero, and every label tells the truth





The kept article: paid by key reveal.

One environment indicator in the header · **each payment step** links to its transaction · **a refund says why**

— Reachability · Feasibility

Agents need facts behind paywalls, and writers need to be paid per read

Who pays

- **The asker,** a person or their agent, pays per article, inside a budget they set per question.
- **The writer** sets a price per article and is paid on the ledger, with an XRPL address and no card processor.
- **We earn a service fee** when another agent hires ResearchAgent over x402; part of it goes to every cited writer.

Market

- **Writers are being left out.** Small publishers have lost search referrals, and AI licensing deals mostly go to major brands.
- **Today's tools price a whole site** per crawl or by revenue share.
- **None we reviewed** judges value before buying, checks delivery against a promise, or refunds.

Why now

- **Agents act through tools:** Claude Code, Codex and other MCP clients call them on their own.
- **x402 gives HTTP a payment step** a machine can complete.
- **XRPL token escrow** is enabled on Testnet, so the same escrow can settle in a stablecoin once issuers allow it.

> **Where we are:** no customers, pilots or revenue yet, and we have not sized the market. Writer number one is us, labelled; the next are people who consent. The service fee and royalty split ran once on Testnet.

Open Markets / CJL, "Same Gatekeepers, New Tollbooths" (Apr 2026) · Cloudflare Pay Per Crawl and x402 posts · docs/FINAL-PUSH.md §14 · docs/evidence/hire-274.md · qa-prep Q4

— Reachability · compliance readiness

Built for audit. Honest about what a regulated launch still needs.

Every spend is authorised by deterministic policy code, capped, and written to a hash-chained audit log whose head is anchored on the XRP Ledger, even when the agent refuses to buy. It is Testnet only, and each gap has a named next step.

	Built (Testnet)	Not done
Custody	Keys stay server-side, out of logs, audit rows and the browser	KMS or HSM and rotation; the hire flow holds funds in transit
Spend limits	A budget per question and a daily cap; only policy code buys	Maker-checker, velocity limits
Audit	Hash-chained log, anchored heads, offline verification	Signed, durable retention; retention periods undecided
Screening, KYB	A payee denylist hook; XRPL Credentials badge for writers	No sanctions provider, no KYB, no Permissioned Domains
Regulation	Questions mapped for MAS (PSA, DPT services), GENIUS Act, Travel Rule	No licence, no legal opinion; mainnet blocked in code

Ranked builder feedback for Ripple, each with a runnable repro

1 **Issuer permission blocks RLUSD escrow**
Token escrow is enabled; the issuer lacks AllowTrustLineLocking. Fix: a network and issuer preflight.

2 **An issued amount must mean the delivered amount**
Only validated delivered_amount proves what arrived. Fix: negative issued-asset test vectors.

3 **A signed IOU quote overflowed the 1 KiB Memo**
Observed on Testnet, 7 Nov. Fix: check the serialized Memo size before signing.

4 **Escrow expiry doesn’t return funds by itself**
Funds stayed locked until an explicit EscrowCancel. Fix: document expiry versus cancellation.

5 **Endpoints disagree on retained history**
One answered “not found” for a validated hash; HTTP lacks browser CORS. Fix: an endpoint matrix.

6 **Settled invoice consumption blocks recovery**
A same-payment retry is refused while delivery still needs recovering. Fix: idempotent replay.

7 **A header-only x402 v2 challenge is not retried**
The client returns the unpaid 402. Fix: read the v2 header first.

\+ **Mainnet readiness: a checklist, not a claim**
Plus what worked well: escrow, delivered_amount, Credentials, fast finality.

> 7 ranked findings with runnable repros · 35 numbered findings (B01–B35) · 15 September submissions (4–5 Sep) · 8 upstream drafts, 0 filed.

— Team · check every claim

A small team, a fleet of coding agents, and a receipt for every claim

Guruprasath

Product and engineering since SingHacks: plans the work, directs the coding agents and owns the merge gate.

Miul

SingHacks build: the research-plan contracts, quote binding and the first test harness.

Andrew Nguyen

Before SingHacks: the idea briefs and data sources.

How we build

Coding agents work in parallel lanes; every merge passes one local gate of tests and the five hard gates. 236 pull requests since 21 Sep.

Check it yourself

Live sandbox, 13–19 Nov	`ra-front-651013803640.asia-southeast1.run.app`
Recorded Testnet run	`collaboration95.github.io/…/demo/`
Ledger page, live checks	`collaboration95.github.io/…/ledger/`
Progress since SingHacks	`collaboration95.github.io/…/progress/`
Release, every artifact	`v1.0.2 on GitHub Releases`
Code and evidence	`github.com/Collaboration95/theFastandtheFungible`
Demo video	linked in the submission form

People and roles from the git history (git shortlog) · AGENTS.md (how work is run) · Testnet XRP and TEST-SGD have no value; receipts keep ledger indexes if Testnet resets